

# Control flow Statements

1. Conditional statement
2. Looping statement
3. Transfer statement

## Conditional Statements

### 1. If statement:

```
If(condition)

{

    Statement 1;

    Statement 2;

}
```

As seen above the way the way if condition work is

1. If condition is true then the statement it will get executed.
2. If condition false then the statement it will not get executed.
3. Every condition at any point of time can have only one status either true or false.
4. To generate Boolean, we need to comparison operators
5. Comparison operators' combination of equality and relational operator.  
Ex: <, >, !=, ==, <=, >=
6. The comparison operator has the ability to generate Boolean values hence these are used in condition.

**Write java program to check if is greater than 100.**

```
1 class if1
2 {
3     public static void main(String [] args)
4     {
5         int num1=150;
6         if(num1>100)
7         {
8             System.out.println("Num1 is greater");
9         }
10    }
11 }
```

### 3. Control Flow statements

Syntax:

```
If(condition)
{
    Statement;
}
else
{
    Statement;
}
```

If the condition is true then the statement within if block will get executed. But if the condition is false the statement within else block will get executed.

**Write java program to check if person is eligible for voting**

```
1  class ifelse1
2  {
3      public static void main (String [] args)
4      {
5          int age=17;
6          if(age>=18)
7          {
8              System.out.println("Eligible for voting");
9          }
10         else
11         {
12             System.out.println("Not Eligible for voting");
13         }
14     }
15 }
```

### 3. Control Flow statements

**Write java program to check out of two numbers which one is greater.**

```
1 class ifelse2
2 {
3     public static void main (String [] args)
4     {
5         int num1=10,num2=20;
6         if(num1>num2)
7         {
8             System.out.println("Num1 is greator");
9         }
10        else
11        {
12            System.out.println("Num2 is greater");
13        }
14    }
15 }
```

**Write java program to check if person is eligible for marriage or not.**

```
1 class ifelse3
2 {
3     public static void main (String [] args)
4     {
5         int age=21;
6         char gender = 'm';
7
8         if(age>=21 && gender=='m')
9         {
10            System.out.println("Eligible for marriage");
11        }
12        else
13        {
14            System.out.println("Not eligible for marriage");
15        }
16    }
17 }
```

### 3. Control Flow statements

#### Truth table

| Cond1 | cond2 | O/P |
|-------|-------|-----|
| F     | F     | F   |
| T     | F     | F   |
| F     | T     | F   |
| T     | T     | T   |

A Single equal two ( = ) is used for assignment or initialization where ( == ) operator is made for compare operations.

```
1 class ifelse4
2 {
3     public static void main (String [] args)
4     {
5         int age=18;
6         char c='f';
7
8         if(age>=18 && c =='f')
9         {
10            System.out.println("Eligible for voting.");
11        }
12        else
13        {
14            System.out.println("Not eligible for voting");
15        }
16    }
17 }
```

If else 3 and 4 combine

```
1 class ifelse34
2 {
3     public static void main (String [] args)
4     {
5         int age=19;
6         char gender='f';
7         if((gender=='m' && age>=21)|| (gender=='f' && age>=18))
8         {
9             System.out.println("Eligible");
10        }
11        else
12        {
13            System.out.println("Not Eligible");
14        }
15    }
16 }
```

### 3. Control Flow statements

**Write java program to check if program or student is eligible for IIT based on if the science or maths percentage is above 90%**

```
1 class ifelse5
2 {
3     public static void main(String [] args)
4     {
5         int science=90, math=80;
6
7         if(science>=90 || math>=90)
8         {
9             System.out.println("Eligible for IIT");
10        }
11        else
12        {
13            System.out.println("NOT Eligible for IIT");
14        }
15    }
16 }
```

### 3. Control Flow statements

## If else – if

Syntax:

```
If(condition)
{
    Statement;
}

Else if(condition)
{
    Statement;
}

Else if(condition)
{
    Statement;
}

Else
{
    Statement;
}
```

**Write java program to check out of two number which one greatest and also check both are same**

```
1 class ifelseif1
2 {
3     public static void main(String [] args)
4     {
5         int a=23,b=23;
6         if(a>b)
7         {
8             System.out.println("Num1 is greater");
9         }
10        else if(b>a)
11        {
12            System.out.println("Num2 is greater");
13        }
14        else if(a==b)
15        {
16            System.out.print("Both number are equal");
17        }
18        else
19        {
20            System.out.println("Invalid Number");
21        }
22    }
23 }
```

### 3. Control Flow statements

Write java program to simulate traffic signal

```
1 class ifelseif2
2 {
3     public static void main(String [] args)
4     {
5         char signal='r';
6         if(signal=='g')
7         {
8             System.out.println("RUN");
9         }
10        else if(signal=='r')
11        {
12            System.out.println("STOP!!!");
13        }
14        else if(signal=='y')
15        {
16            System.out.println("SLOW DOWN BABY");
17        }
18    }
19 }
```

Write java program to simulate a banking domain

```
1 class ifelseif3
2 {
3     public static void main (String [] args)
4     {
5         char custreq='c';
6         if(custreq=='c')
7         {
8             System.out.println("Desposit the money");
9         }
10        else if(custreq=='d')
11        {
12            System.out.println("Withdraw the money");
13        }
14        else if (custreq=='s')
15        {
16            System.out.println("generate the statement");
17        }
18    }
19 }
20 }
```

### 3. Control Flow statements

## Nested if else

Syntax:

```
If(condition)
{
    If(condition)
    {
        statement;
    }
    Else
    {
        statement;
    }
}
Else
{
    If(condition)
    {
        statement;
    }
    Else
    {
        statement;
    }
}
```

### 3. Control Flow statements

Write java program to check whether product is available or not if it is available and quantity above file then purchase it.

---

```
1  class nestedifelse1
2  {
3      public static void main (String [] args)
4      {
5          boolean motostatus=false;
6          int QtyToBuy=5;
7          //motostatus =true
8          //if qty=10 buy motorola else wait
9          //false and if qty>10 buy poco
10     if(motostatus==true)
11     {
12         if(QtyToBuy==10)
13         {
14             System.out.println("Buy the motorola");
15         }
16         else
17         {
18             System.out.println("Wait for motorola");
19         }
20     }
21     else
22     {
23         if(QtyToBuy>=10)
24         {
25             System.out.println("Buy Poco");
26         }
27         else
28         {
29             System.out.println("Wait for Poco");
30         }
31     }
32 }
33 }
```

### 3. Control Flow statements

## Switch statement

#### Syntax:

```
Switch(variable / argument / parameter)

{
    case caselabel: statement;

    Case caselabel: statement;

    Default: statement;

}
```

#### Write a java program to implement switch statement

---

```
1  class Switch1
2  {
3      public static void main(String [] args)
4      {
5          int a=2;
6          switch(a)
7          {
8              case 1: System.out.println("Inside first statement");
9                  break;
10             case 2: System.out.println("Inside second statement");
11                 break;
12             case 3: System.out.println("Inside third statement");
13                 break;
14             }
15         }
16     }
```

The switch statement works on a concept of case matching

In the switch statements it matches the arguments with the case label, whenever it matches it will execute the statements only of respective case.

**Break statement:** It is meant to break the control within the switch and hence it will move control outside the switch statement.

### 3. Control Flow statements

If the switch argument is not matching any of the case label, then by default it will match default case.

```
1  class Switch2
2  {
3      public static void main(String [] args)
4      {
5          int a=3;
6          switch(a)
7          {
8              case 1: System.out.println("Inside first statement");
9                  break;
10             case 2: System.out.println("Inside second statement");
11                 break;
12             default : System.out.println("Inside third statement");
13                 break;
14         }
15     }
16 }
```

Break statements are not mandatory

Conclusion: while using switch statements always use break statement.

```
1  class Switch3
2  {
3      public static void main(String [] args)
4      {
5          int a=1;
6          switch(a)
7          {
8              case 1: System.out.println("Inside first statement");
9                  break;
10             case 2: System.out.println("Inside second statement");
11                 break;
12             default: System.out.println("Inside default statement");
13                 break;
14         }
15     }
16 }
```

### 3. Control Flow statements

Cases are not mandatory with switch statement

```
1 class Switch4
2 {
3     public static void main (String [] args)
4     {
5         int a=1;
6         switch(a)
7         {
8             default: System.out.println("Inside default statement..");
9         }
10    }
11 }
```

Default case is also not mandatory

```
1 class Switch5
2 {
3     public static void main (String [] args)
4     {
5         int a=3;
6         switch(a)
7         {
8             case 1: System.out.println("Inside statement one.");
9             break;
10            case 2: System.out.println("Inside statement two.");
11            break;
12        }
13    }
14 }
```

What is the output of following code class

```
1 class Switch6
2 {
3     public static void main (String [] args)
4     {
5         int a=3;
6         switch(a)
7         {
8             }
9         System.out.println("Stop");
10    }
11 }
12
13 }
```

Conclusion: code will compile and execute because cases and default is not mandatory

### 3. Control Flow statements

Statement should be either part of cases or default else it will throw an error

```
1 |class Switch7
2 |{
3 |    public static void main (String [] args)
4 |    {
5 |        int a=3;
6 |        switch(a)
7 |        {
8 |            System.out.println("Inside Switch");
9 |        }
10 |    }
11 |}
```

This code will throw an error.

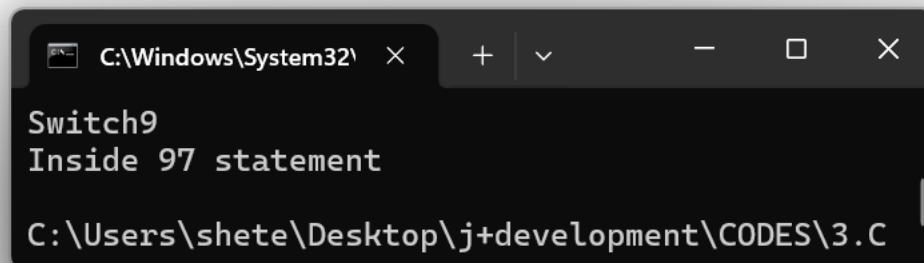
**What are valid and invalid argument in switch statement**

**Valid** : int, short, byte, char, String

**Invalid:** float, long, double, Boolean

**What is output of the following code?**

```
1 |class Switch9
2 |{
3 |    public static void main (String [] args)
4 |    {
5 |        char var='a';
6 |        switch(var)
7 |        {
8 |            case 96: System.out.println("Inside");
9 |            break;
10 |           case 97: System.out.println("Inside 97 statement");
11 |           break;
12 |           default: System.out.println("Default statement");
13 |           break;
14 |        }
15 |    }
16 |}
17 |
18 |}
```



```
C:\Windows\System32\
Switch9
Inside 97 statement
C:\Users\shete\Desktop\j+development\CODES\3.C
```

### 3. Control Flow statements

```
1  class Switch10
2  {
3      public static void main (String [] args)
4      {
5          int x='b';
6          switch(x)
7          {
8              case 97:System.out.println("Inside");
9              break;
10             case 98:System.out.println("Inside 97 statement");
11             break;
12             default:System.out.println("Default statement");
13             break;
14         }
15     }
16 }
17
18 }
```

Duplicate case labels are not allowed in switch

The compiler throws an error for the below program because 99 with the help of Unicode character representation can be represented as character 'c' as well as character 'c' with the help of Unicode can be represented in the form of integer.

Based on the switch argument we can pass, the case label will be of same type else it will throw an error.

```
1  class Switch12
2  {
3      public static void main (String []args)
4      {
5          String s="one";
6          switch(s)
7          {
8              case "one":System.out.println("I am one statement");
9              break;
10             case 1:System.out.println("i am 1 statement");
11             break;
12             default:System.out.println("i am defalut");
13             break;
14         }
15     }
16 }
```

### 3. Control Flow statements

The case labels should be within the range based on the type of switch arguments

```
1  class Switch13
2  {
3      public static void main (String [] args)
4      {
5          byte var=127;
6          switch(var)
7          {
8              case 126: System.out.println("I am 126 S");
9                      break;
10             case 127: System.out.println("I am 127 S");
11                     break;
12             case 128: System.out.println("I am 125 S");
13                     break;
14             default: System.out.println("I am default statement");
15                     break;
16         }
17     }
18 }
19 //Switch13.java:12: error: incompatible types: possible lossy conversion from int to byte
20
```

### 3. Control Flow statements

## Constant

1. To make variable behaviour as constant we need prefix keyword final before variable.
2. A constant can be assigned only once if try to initialize or assign second time it will throw  
`//final`

## Looping statement

1. Loops are used to execute defined statement for predefined number of types.
2. The loop helps us to reuse the statement and also help reduce length of code which help consuming lesser memory which is increase speed application.
3. There are mainly three type of loop
  - For
  - While
  - Do while

## For loop

For loop can be further categorized into three types

1. For I
2. Nested for
3. For each>>> array collection

Syntax:

`For (declaration/initialization, condition, increment/decrement)`

```
{    statement;  
    Statement;  
  
}
```

### 3. Control Flow statements

#### Increment and Decrement operators

- These operators are used to increment or decrement of a variable.
- These operators perform operation like increment or decrement with respect to value.
- These operators can further be categorized into two types
  - Pre: increment and decrement
  - Post: increment and decrement
- For pre increment operator it will first increment the value and only then it will perform the operation.
- With the help of post increment operator, we will first perform operation (print) and then it will perform increment.

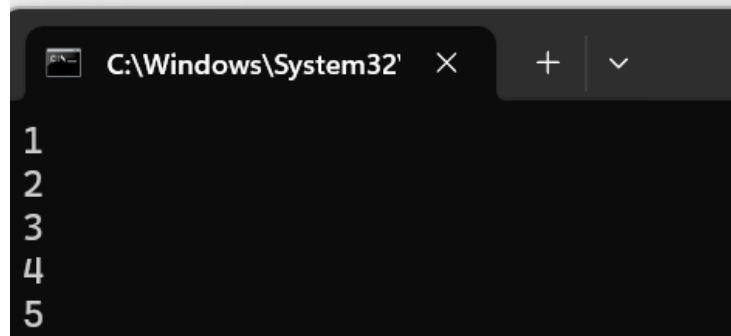
//increment1

//decrement1

### 3. Control Flow statements

**Write java program to print value from 1 to 3 using for loop**

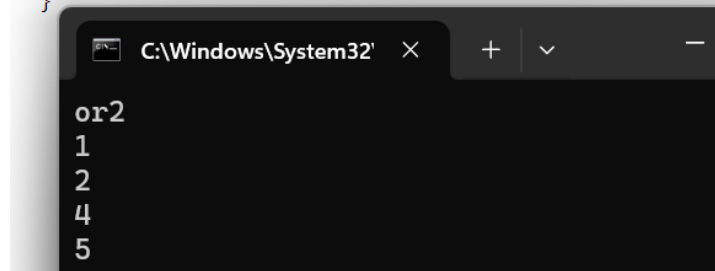
```
//for loop print 1 to 5\nclass For1\n{\n    public static void main (String [] args)\n    {\n        for(int i=1;i<=5;i++)\n        {\n            System.out.println(i);\n        }\n    }\n}
```



- Declaration and initialization only happen once.
- The condition is checked based on total number of iterations plus times (3+1)
- The increment and decrement will happens based on total number of iterations.

**Write java program to check whether number 3 is present between one 1 to 5 numbers.**

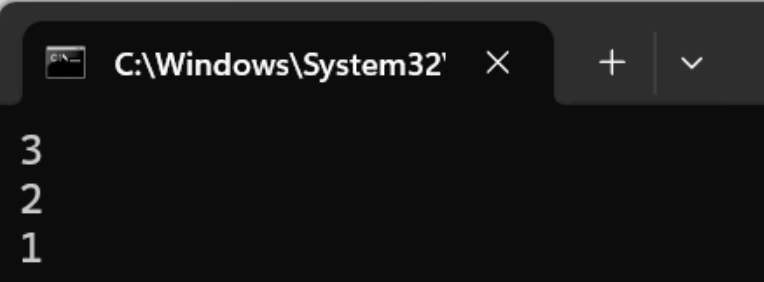
```
//for loop for printing number one to five and skip if\n 3\nclass For2\n{\n    public static void main (String [] args)\n    {\n        for (int i=1;i<=5 ;i++ )\n        {\n            if(i==3)\n            {\n                continue;\n            }\n            System.out.println(i);\n        }\n    }\n}
```



### 3. Control Flow statements

Write java program to display number from 1 to 3 in descending order.

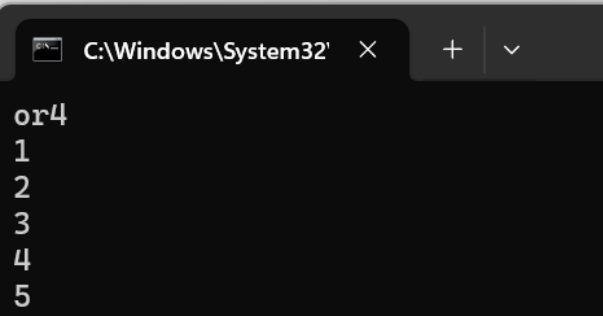
```
//for (i=3;i>=1;--i)and print i
class For3
{
    public static void main(String [] args)
    {
        for (int i=3;i>=1 ;--i )
        {
            System.out.println(i);
        }
    }
}
```



#### Declaration as well as initialization

Declaration and initialization is not mandatory within for statement.

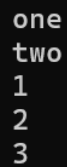
```
// initialize outside and give conditions and
class For4
{
    public static void main (String [] args)
    {
        int i=1;
        for (;i<=5 ;i++ )
        {
            System.out.println(i);
        }
    }
}
```



### 3. Control Flow statements

**We can use single statement on multiple places of declaration as well as initialization.**

```
class For5
{
    public static void main (String [] args)
    {
        int i=1;
        for (System.out.println("one"), System.out.println("two"); i<=3 ; i++ )
        {
            System.out.println(i);
        }
    }
}
```



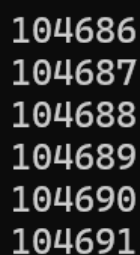
```
one
two
1
2
3
```

- If a variable is declared within the for loop it can be accessed only within the for loop but if a variable is declared within the main then it can be accessed within the complete main area.

#### Conditions

1. Condition is not mandatory
2. If we do not set the condition then the compiler will set the default condition true
3. Statements are now allowed in place of condition.

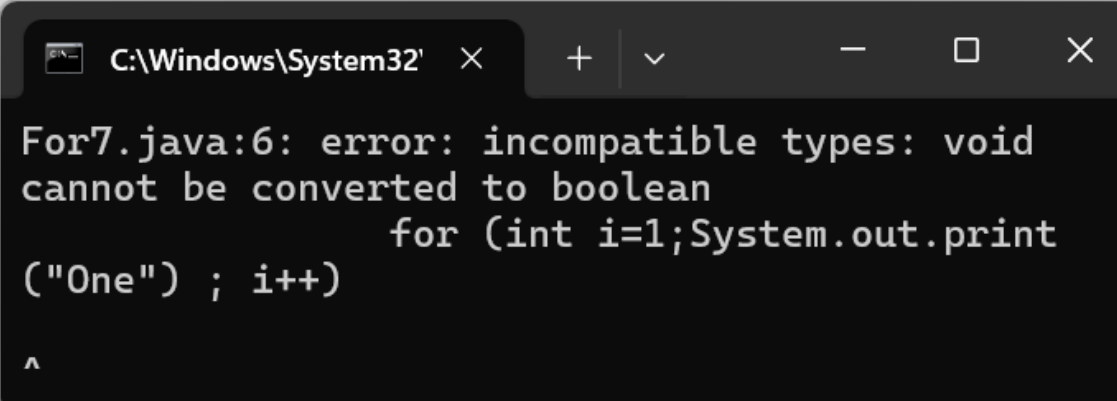
```
//dont give condition and print i
class For6
{
    public static void main (String [] args)
    {
        for (int i=1; ; i++ )
        {
            System.out.println(i);
        }
    }
}
//OP infinity
```



```
104686
104687
104688
104689
104690
104691
```

### 3. Control Flow statements

```
//sopi in place of condition and print i
class For7
{
    public static void main (String [] args)
    {
        for (int i=1;System.out.print("One") ; i++)
        {
            System.out.println(i);
        }
    }
}
```



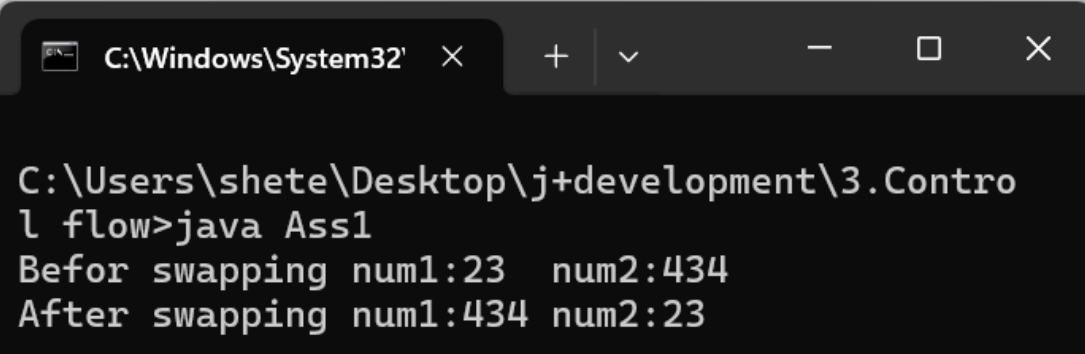
C:\Windows\System32' x + v - □ ×

For7.java:6: error: incompatible types: void  
cannot be converted to boolean  
for (int i=1;System.out.print  
("One") ; i++)

^

Write java program to swap values of two variables without using 3<sup>rd</sup> variable.

```
class Ass1
{
    public static void main (String [] args)
    {
        int num1=23,num2=434;
        System.out.println("Befor swapping num1:"+num1+"    num2:"+num2);
        num1=num1+num2;//addition of both numbers stored in num1
        num2=num1-num2;//subtracting num2 from total
        num1=num1-num2;
        System.out.println("After swapping num1:"+num1+"    num2:"+num2);
    }
}
```



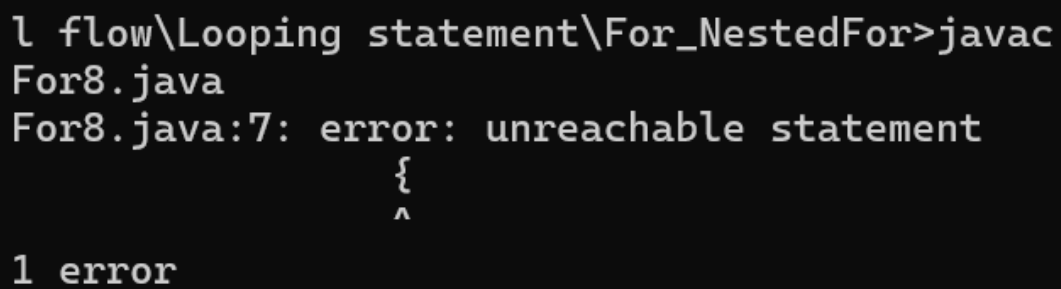
C:\Windows\System32' x + v - □ ×

C:\Users\shete\Desktop\j+development\3.Contro  
l flow>java Ass1  
Befor swapping num1:23 num2:434  
After swapping num1:434 num2:23

### 3. Control Flow statements

What is output of following code

```
class For8
{
    public static void main (String [] args)
    {
        for (int i=1;false ;i++ )
        {
            System.out.println(i);
        }
    }
}
```



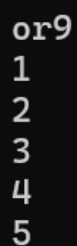
C:\Windows\System32' x + v - □ X

```
l flow\Looping statement\For_NestedFor>javac
For8.java
For8.java:7: error: unreachable statement
        {
        ^
1 error
```

Output is error (unreachable statement) as a condition is false it will never reach the statement.

- Increment and decrement is not mandatory

```
class For9
{
    public static void main (String [] args)
    {
        for(int i=1;i<=5;)
        {
            System.out.println(i);
            i++;
        }
    }
}
```



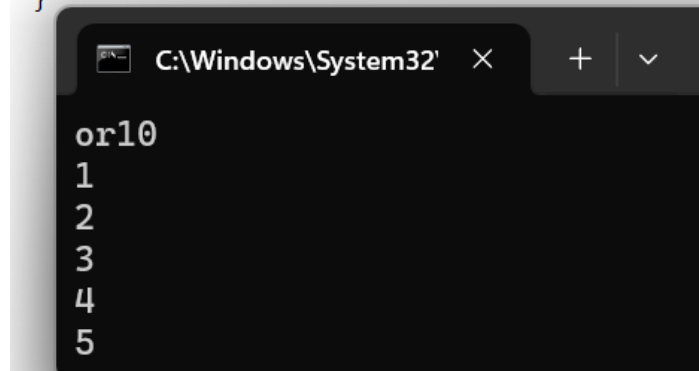
C:\Windows\System32' x + v - □ X

```
or9
1
2
3
4
5
```

### 3. Control Flow statements

What is output of the following code

```
class For10
{
    public static void main (String [] args)
    {
        int i=1;
        for(;i<=5;)
        {
            System.out.println(i);
            i++;
        }
    }
}
```

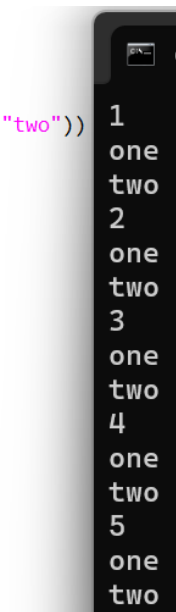


A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32'. The command prompt displays the output of the 'For10' program, which is the numbers 1 through 5, each on a new line.

```
or10
1
2
3
4
5
```

Single statements or multiple statements are allowed in place of increment and decrement.

```
class For11
{
    public static void main (String [] args)
    {
        for(int i=1;i<=5;System.out.println("one"),System.out.println("two"))
        {
            System.out.println(i);
            i++;
        }
    }
}
```



A screenshot of a Windows command prompt window showing the output of the 'For11' program. The output consists of five lines, each containing the number 1, 2, 3, 4, or 5, followed by the words 'one' and 'two' on the same line, separated by a space.

```
1 one two
2 one two
3 one two
4 one two
5 one two
```

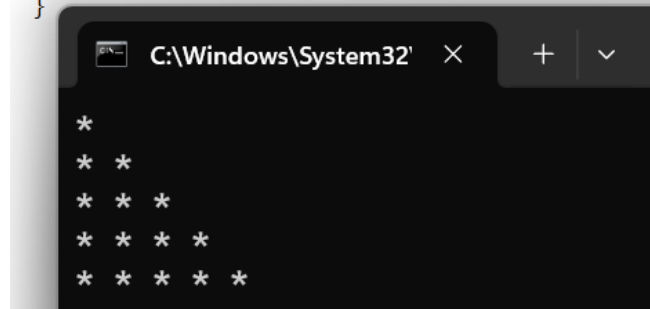
### 3. Control Flow statements

```
1  class For12
2  {
3      public static void main (String [] args)
4      {
5          for(;;)
6          {
7              System.out.println("Hello");
8          }
9      }
10 } //output:Hello x infinity|
```

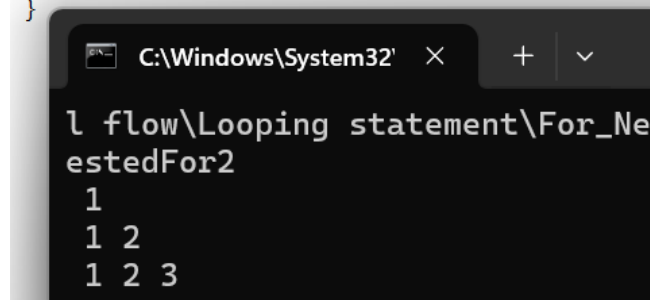
### 3. Control Flow statements

#### Nested For

```
class NestedFor1
{
    public static void main (String [] args)
    {
        for(int i=1;i<=5;i++)
        {
            for(int j=1;j<=i;j++)
            {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

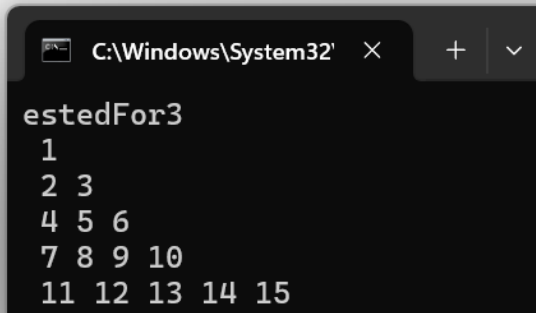


```
class NestedFor2
{
    public static void main (String [] args)
    {
        for(int i=1;i<=3;i++)
        {
            for(int j=1;j<=i;j++)
            {
                System.out.print(" "+j);
            }
            System.out.println();
        }
    }
}
```



### 3. Control Flow statements

```
class NestedFor3
{
    public static void main (String [] args)
    {
        int x=1;
        for(int i=1;i<=5;i++)
        {
            for(int j=1;j<=i;j++)
            {
                System.out.print(" "+x);
                x++;
            }
            System.out.println();
        }
    }
}
```



```
C:\Windows\System32' X + v
NestedFor3
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

### 3. Control Flow statements

#### While

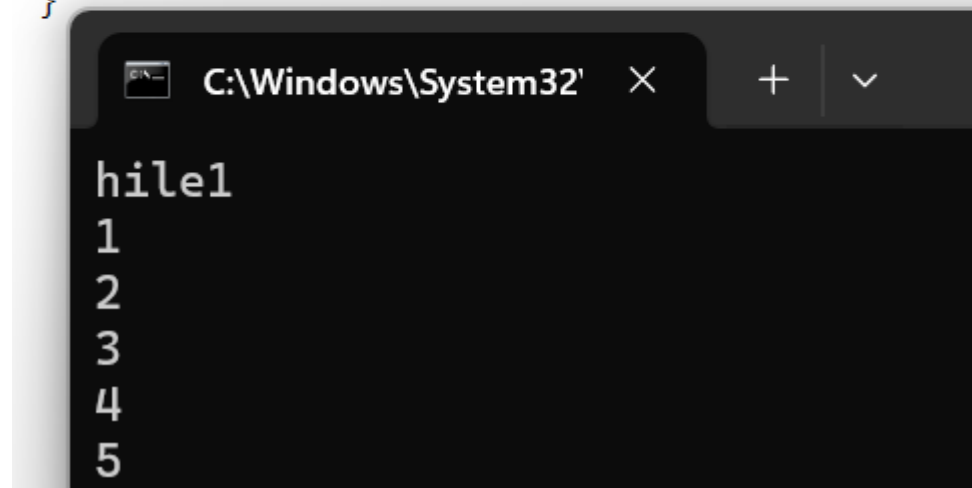
Syntax:

While(condition)

```
{    Statement1;
    Statement2;
}
```

**Write java program to print values from 1 to 5 using while loop.**

```
class While1
{
    public static void main (String [] args)
    {
        int i=1;
        while(i<=5)
        {
            System.out.println(i);
            i++;
        }
    }
}
```



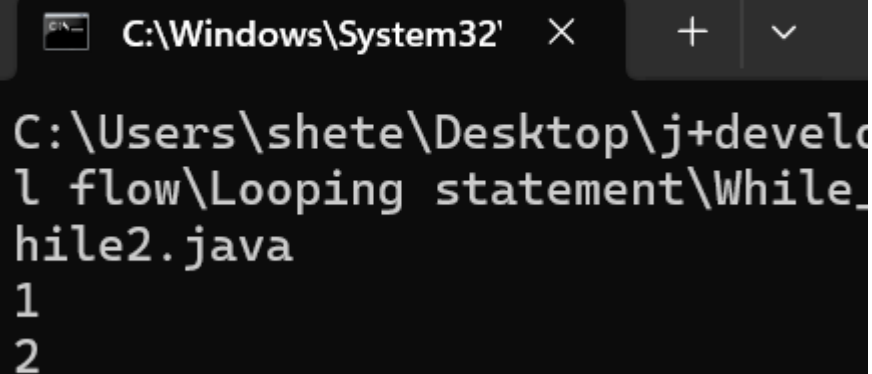
The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command prompt displays the output of the Java program, which is the numbers 1 through 5, each on a new line. The output is preceded by the label 'hile1'.

```
hile1
1
2
3
4
5
```

### 3. Control Flow statements

Write java program to iterate from 1 to 5 in ascending order and check if number 3 is present.

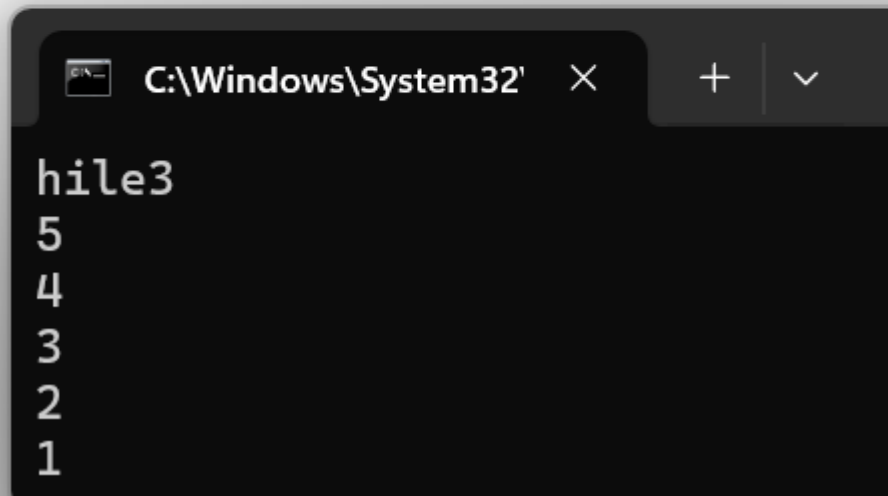
```
class While2
{
    public static void main (String [] args)
    {
        int i=1;
        while(i<=5)
        {
            if(i==3)
            {
                continue;
            }
            System.out.println(i);
            i++;
        }
    }
}
```



```
C:\Windows\System32' X + v
C:\Users\shete\Desktop\j+develo
l flow\Looping statement\While_
hile2.java
1
2
```

### 3. Control Flow statements

```
class While3
{
    public static void main (String [] args)
    {
        int i=5;
        while (i>=1)
        {
            System.out.println(i);
            --i;
        }
    }
}
```



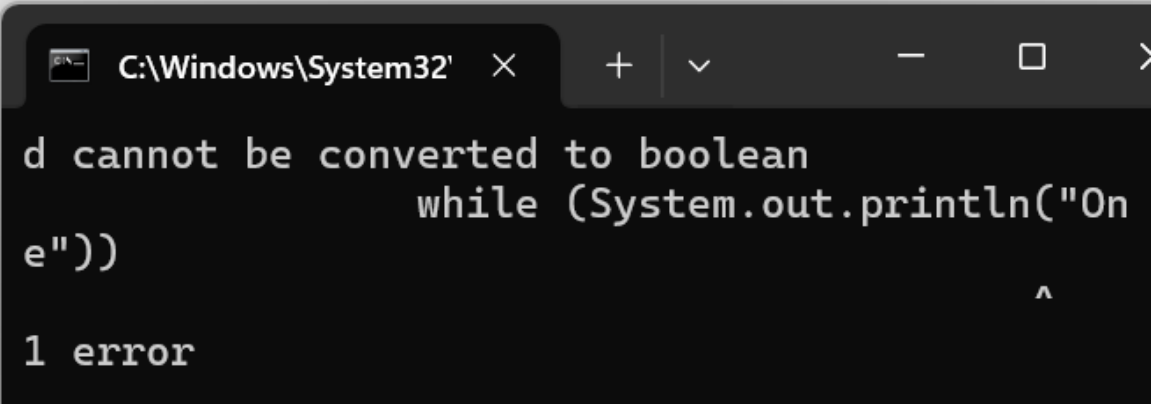
A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32' and standard window controls. The command prompt displays the output of the Java program, which is the numbers 5, 4, 3, 2, and 1, each on a new line. The text 'hile3' is also visible on the first line of the output.

```
hile3
5
4
3
2
1
```

### 3. Control Flow statements

**We cannot use statement in place of conditions**

```
class While4
{
    public static void main (String [] args)
    {
        int i=1;
        while (System.out.println("One"))
        {
            System.out.println(i);
            i++;
        }
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command prompt displays the following error message:

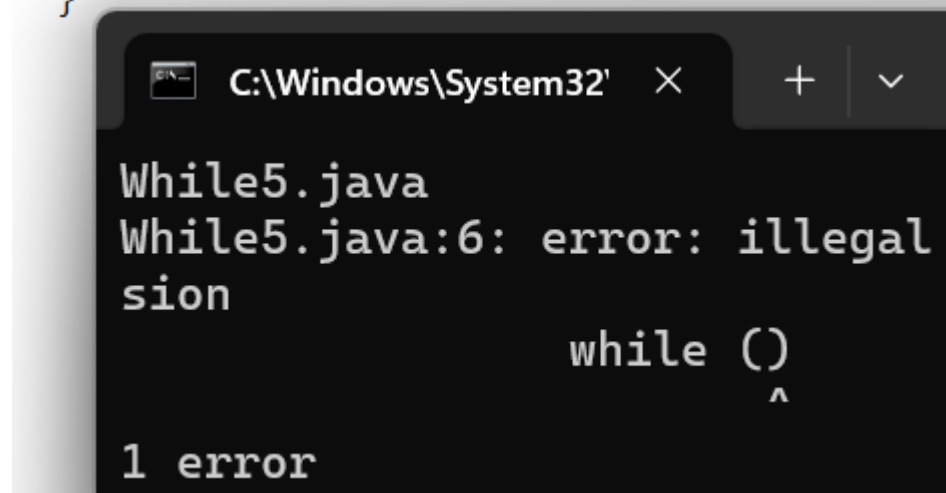
```
d cannot be converted to boolean
        while (System.out.println("On
e"))
^
1 error
```

### 3. Control Flow statements

Write java program to print number from 1-5 in descending order using while loop

If we do not set any condition in while loop then the compiler will not set any default condition.

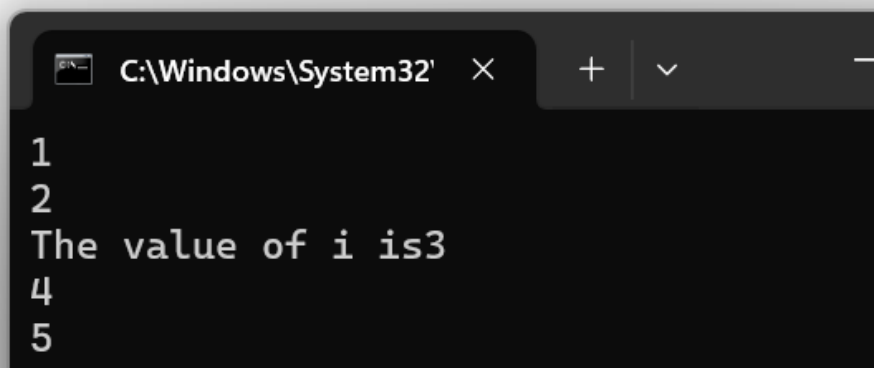
```
class While5
{
    public static void main (String [] args)
    {
        int i=-1;
        while ()
        {
            System.out.println(i);
            i++;
        }
    }
}
```



### 3. Control Flow statements

```
class While6
{
    public static void main (String [] args)
    {
        int i=1;
        while(i<=5)
        {
            i--;
            System.out.println(i);
            i++;
        }
    }
} //0 x infinity
```

```
class While7
{
    public static void main(String [] args)
    {
        int i=1;
        while(i<=5)
        {
            if(i==3)
            {
                System.out.println("The value of i is"+i);
                i++;
                continue;
            }
            System.out.println(i);
            i++;
        }
    }
}
```



```
C:\Windows\System32' x + v -
1
2
The value of i is 3
4
5
```

The for loop and while loop is meant to check the condition at the start hence such loops are called as entry-controlled loops.

### 3. Control Flow statements

#### Do-While loops

Syntax:

Do

{

Statement;

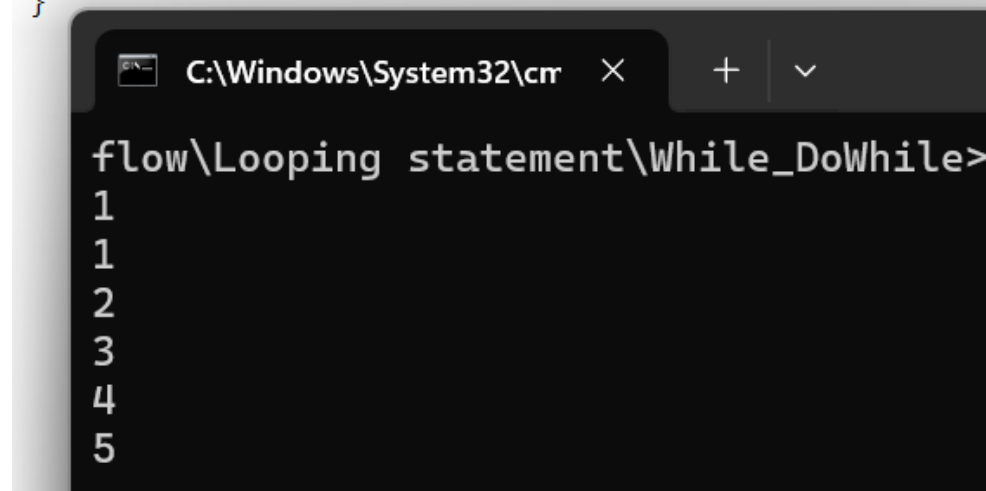
}

While(condition);

In do while loop condition is checked at the end hence such loops are called as exit control loops

**Write java program to print numbers from 1 to 5 using do while loop**

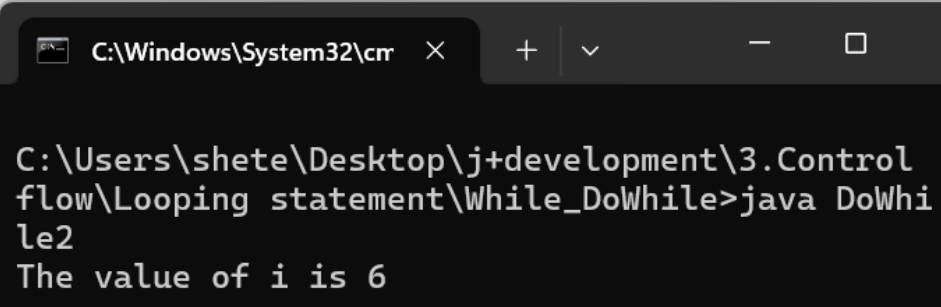
```
class Dowhile1
{
    public static void main (String [] args)
    {
        int a=1;
        do
        {
            System.out.println("The value of i is "+a);
            a++;
        }
        while(a<=5);
    }
}
```



```
C:\Windows\System32\cmd
flow\Looping statement\While_DoWhile>
1
1
2
3
4
5
```

### 3. Control Flow statements

```
class Dowhile2
{
    public static void main (String [] args)
    {
        int i=6;
        do
        {
            System.out.println("The value of i is "+i);
            i++;
        }
        while(i<=5);
    }
}
```

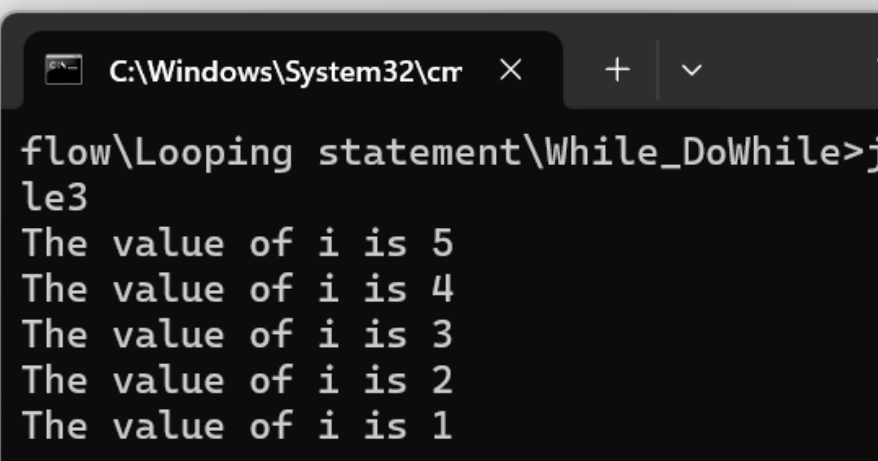


```
C:\Windows\System32\cmd.exe
C:\Users\shete\Desktop\j+development\3.Control
flow\Looping statement\While_DoWhile>java Dowhi
le2
The value of i is 6
```

Regardless do while condition is true or false the statement within do while will at least get executed once

**Write java program to print number from 1-5 in descending order.**

```
class Dowhile3
{
    public static void main (String [] args)
    {
        int i=5;
        do
        {
            System.out.println("The value of i is "+i);
            i--;
        }
        while(i>=1);
    }
}
```



```
C:\Windows\System32\cmd.exe
flow\Looping statement\While_DoWhile>java Dowhile3
The value of i is 5
The value of i is 4
The value of i is 3
The value of i is 2
The value of i is 1
```

### 3. Control Flow statements

In the do while loop also if we do not set the condition the compiler will not set the condition which means condition is mandatory.

```
class Dowhile4
{
    public static void main (String [] args)
    {
        int i=5;
        do
        {
            System.out.println("The value of i is "+i);
            i--;
        }
        while();
    }
}
```

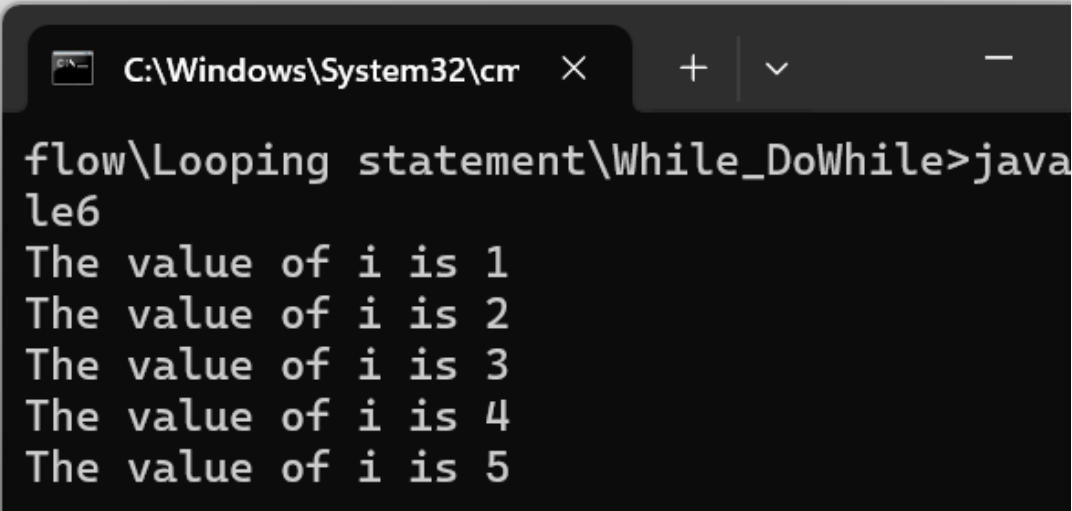
```
flow\Looping statement\While_DoWhile>javac DoWh
ile4.java
DoWhile4.java:11: error: illegal start of expres
sion
                while();
                  ^
1 error
```

```
class Dowhile5
{
    public static void main (String [] args)
    {
        int i=1;
        do
        {
            System.out.println("The value of i is "+i);
        }
        while(i<=5);
    }
}
```

[illegible]

### 3. Control Flow statements

```
class DoWhile6
{
    public static void main (String [] args)
    {
        int i=1;
        do
        {
            if(i==3)
            {
                System.out.println("The value of i is "+i);
                i++;
                continue;
            }
            System.out.println("The value of i is "+i);
            i++;
        }
        while(i<=5);
    }
}
```



```
flow\Looping statement\While_DoWhile>java le6
The value of i is 1
The value of i is 2
The value of i is 3
The value of i is 4
The value of i is 5
```

### 3. Control Flow statements

# Transfer Statements

## 1.goto

These statements are used to move the control from one part of program to another part of program. It is not considered as good coding practice but it is still supported by Java.

## 2.return

Method

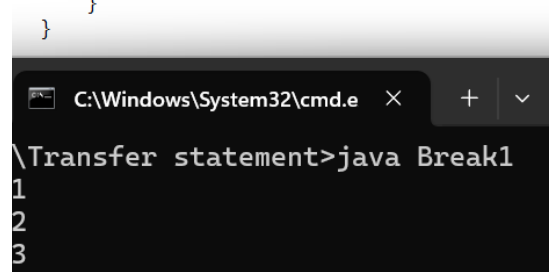
## 3.try-catch

Exception handling

## 4.break

It is meant to break the control either from switch or from loop. It is not meant to break condition.

```
class Break1
{
    public static void main (String [] args)
    {
        for(int i=1;i<5;i++)
        {
            System.out.println(i);
            if(i==3)
            {break;}
        }
    }
}
```

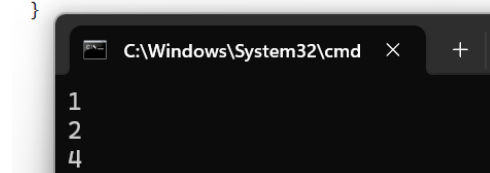


```
C:\Windows\System32\cmd.e  X  +  v
\Transfer statement>java Break1
1
2
3
```

## 5.continue

The continue statement acts as a skip statement where, whenever the continue statement is encountered it will skip all the statements and move the control at the end of the loop.

```
class Continue1
{
    public static void main (String [] args)
    {
        for(int i=1;i<5;i++)
        {
            if(i==3)
            {continue;}
            System.out.println(i);
        }
    }
}
```



```
C:\Windows\System32\cmd  X  +
1
2
4
```

### 3. Control Flow statements

#### 1. When should we go for For loop and while loop

We use a **for loop** when the **number of iterations is known beforehand**. It is best suited for counter-controlled loops such as iterating over arrays, collections, or executing a block of code a fixed number of times.

```
for (int i = 0; i < 5; i++) {  
  
    System.out.println(i);  
  
}
```

We use a while loop when the number of iterations is not known in advance and the loop execution depends on a condition. It is commonly used in scenarios like user input validation, reading data until a condition is met, or menu-driven programs.

Example: for printing list

#### 2. What does a break statement do?

- The **break statement** is used to **immediately terminate the current loop or switch statement** and transfer control to the **next statement after it**.
- When used inside a **loop (for, while, do-while)**, it stops further iterations, even if the loop condition is still true.

#### 3. What will be the out put of the following code class

```
1 class Switch6  
2 {  
3     public static void main (String [] args)  
4     {  
5         int a=3;  
6         switch(a)  
7         {  
8  
9         }  
10        System.out.println("Stop");  
11    }  
12  
13 }
```

#### 4. What are valid and Invalid argument in switch statement

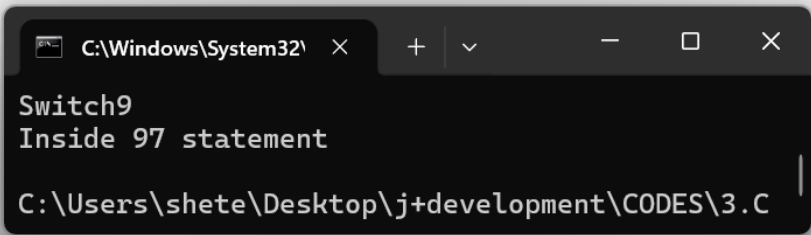
**Valid** : int, short, byte, char, String

**Invalid**: float, long, double, Boolean

### 3. Control Flow statements

#### 5. What will be the output of the following code

```
1 class Switch9
2 {
3     public static void main (String [] args)
4     {
5         char var='a';
6         switch(var)
7         {
8             case 96: System.out.println("Inside");
9             break;
10            case 97: System.out.println("Inside 97 statement");
11            break;
12            default: System.out.println("Default statement");
13            break;
14        }
15    }
16 }
17
18 }
```



```
Switch9
Inside 97 statement
```

#### 6. What will happen if don't set any condition in a for loop

If we **do not set any condition in a for loop**, the condition is treated as **always true**, so the loop becomes an **infinite loop**, unless we explicitly terminate it using break, return, or throw.

```
1 class For12
2 {
3     public static void main (String [] args)
4     {
5         for(;;)
6         {
7             System.out.println("Hello");
8         }
9     }
10 } //output: Hello x infinity
```

#### 7. Write java program to swap values of two variables without using 3<sup>rd</sup> variable.

#### 8. What are the entry-controlled loops.

- **Entry-controlled loops** are loops in which the **condition is checked before entering the loop body**. If the condition is **false initially**, the loop body will **not execute even once**.
- In Java, **for loop** and **while loop** are **entry-controlled loops**

### 3. Control Flow statements

#### 9. What will happen if I don't add break statement in switch

- If we **do not add a break statement in a switch case**, Java exhibits **fall-through behavior**.
- This means that once a matching case is found, **all subsequent cases will execute**, regardless of their case values, **until a break or the end of the switch block is reached**.

```
int x = 1;
switch (x) {
    case 1:
        System.out.println("One");
    case 2:
        System.out.println("Two");
    case 3:
        System.out.println("Three");
}
```

#### Output:

sql

One

Two

Three

